

Thema: OOP-Projekt (Smarte LED-Lampe)

Fach: Informatik 2

Autor: Yannik Markus Lorösch

Matrikelnummer: 88520

Semester: SoSe24

Vorgelegt beim Dozenten am: 12.07.2024

1 Inhaltsverzeichnis

1	Inhaltsverzeichnis	2
2	Einleitung	3
3	Elektronik	4
4	Network Structure	6
5	Software Master	7
6	Software Slave	11
7	Probleme und Verbesserungen	17
8	Fazit	17
9	Quellenverzeichnis	18
10	Abbildungsverzeichnis	19
11	Anhang	20

2 Einleitung

Im Rahmen des Informatik 2 Projektes haben wir die Aufgabe bekommen die Objekt-orientierte Programmierung in Verbindung von Hardware anzuwenden und zu vertiefen. Da wir in unserer Gruppe bereits Kompetenzen in CAD und entwerfen von Schaltungen vorweisen können, entschieden wir uns recht schnell dazu eigene Hardware und ein dazu passendes CAD zu designen. Die Frage war nur was wollen wir umsetzen? Zu Beginn war die Überlegung, ob wir uns an einem vier Achs Roboterarm versuchen. Doch für die Umsetzung der Transformationsmatrix z.B. in Code stellte sich als deutlich anspruchsvoller heraus, als gedacht. Nach langer Überlegung entschieden wir uns am Ende dazu, eine LED-Lampe mit verbauten Sensoren zu konstruieren.

Für die Umsetzung des Projektes teilten wir die Aufgaben für die Konstruktion so auf, dass Yannik die Elektronik und das PCB erstellt und Justin sich um das CAD und die Vorführwand kümmert.

In dieser Dokumentation werden wir uns der Elektronik, dem Netzwerkprotokoll und der Software des Masters und des Slaves widmen. Zusätzlich werden noch die Probleme erläutert und Vorschläge zur Lösung aufgezeigt. Bitte beachten Sie aufgrund des doch relativ großen Umfangs der Software werden nur die wichtigsten Funktionspunkte betrachtet.

3 Elektronik

Als erstes möchte ich gerne auf die Elektronik eingehen diese wurde von mir entwi-



kelt. Zuerst haben wir uns nach einer LED umgesehen welche wir benutzen wollen, dabei sind wir auf eine 6x1,6 W LED ¹gestoßen mit den Farben rot, grün, blau, weiß, orange und ultra-violett. Diese LED wurde von uns mit einer Konstant Stromquelle angesteuert diese wurde wiederum mit PWM von einem Microcon-

troller angesteuert. Die Konstantstromquellen wurden auf 300mA je LED eingestellt diese sind vom Typ CN5711². Als nächstes haben wir uns überlegt welche Sensoren wir den verbauen sollten dabei sind wir aufn Temperatur und Luftfeuchtigkeits-Sensor AHT20 ³sowie den TVOC (Total Volatile Organic Compounds) Sensor AGS10 ⁴und den Licht-Sensor LTR-308 ⁵gekommen. Der Lichtsensor ist besonders gut auf, dass menschliche Sehen abgestimmt und gibt somit einen besonders guten Wert für die Helligkeit aus. Alle Sensoren werden mit I2C von unserem Microcontroller angesteuert. Des Weiteren wurde eine Debug-LED, ein Buzzer mit seiner Ansteuerung und Schutz elektronik verbaut. Des Weiteren wurde ein USB-Anschluss mit Schutzbeschaltung verbaut. Auch wurde an die Schutzbeschaltung aller anderen von außen zugänglichen Anschlüssen gedacht z.B. mit ESD-Dioden. Als Microcontroller kommt ein ESP32-S3 ⁶mit 8MB Flash und 2MB PSRAM zum Einsatz. Dieser wurde mit Micro Python⁷ in der Version 1.22.2 programmiert. Nachdem das Layout der Platine erstellt worden wurden die Platinen in China zur Produktion aufgegeben und dann von uns bestückt. Dabei war vor allem auf die Sensoren achtzugeben da diese sehr schnell beschädigt werden können.

¹<https://www.steinigke.de/download/E6508726-Datasheet-99282-led-hcl-10w-rgbwauv-pix-6-hcl-tx-bgrwpy10b140-001c-LED-Datenblatt-en.pdf>

²https://www.lcsc.com/datasheet/lcsc_datasheet_1809140110_ShangHai-Consonance-Elec-CN5711_C35634.pdf

³https://www.lcsc.com/datasheet/lcsc_datasheet_2310231158_Aosong--Guangzhou-Elec-AHT20_C2757850.pdf

⁴https://www.lcsc.com/datasheet/lcsc_datasheet_2310231158_Aosong--Guangzhou-Elec-AHT20_C2757850.pdf

⁵ https://www.lcsc.com/datasheet/lcsc_datasheet_2109180930_Lite-On-LTR-308ALS-01_C492382.pdf

⁶https://www.lcsc.com/datasheet/lcsc_datasheet_2206281830_Espressif-Systems-ESP32-S3-WROOM-1-N8R2_C2913204.pdf

⁷ https://micropython.org/download/ESP32_GENERIC_S3/

Nach dem Bestücken erfolgte der erste Test der Platinen, welcher sich zum Glück als fast 100% Erfolg herausstellte, nun konnte mit der Entwicklung der Software begonnen werden.

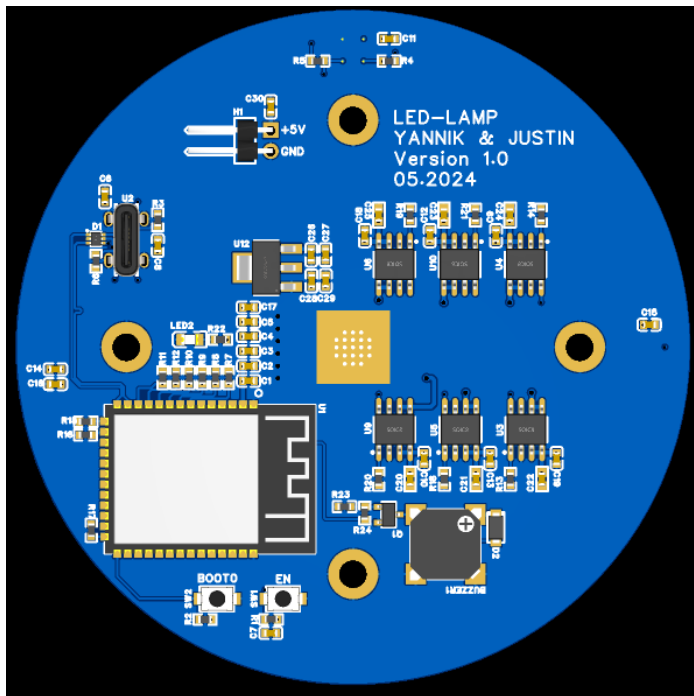


Abbildung 1 Platine Unterseite

Gut ersichtlich ist hier der Prozessor im unteren linken Bereich, sowie der USB-Port darüber. Rechts befinden sich die 6 Konstant-Stromquellen für die 6 Farben der LED, darunter der Buzzer. Links oben befindet sich der Stromstecker für die Externe-Stromversorgung.

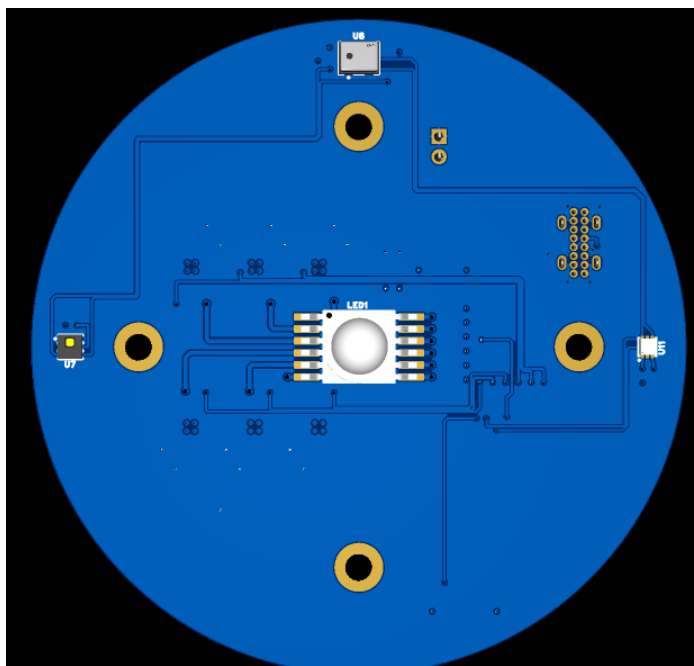


Abbildung 2 Platine Oberseite

Auf der Oberseite befindet sich die LED in der Mitte, gerade darüber der TVOC-Sensor, rechts der Temperatur und Luftfeuchtigkeits-Sensor und am linken Rand der Lichtsensor.

4 Network Structure

Wir haben uns überlegt wie wir die Netzwerk Struktur aufbauen wollen und haben uns für ein Master/Slave Netzwerk entschieden in welchem Datenpakete mit definierter Länge und Struktur übertragen werden. Natürlich haben wir uns auch überlegt wie man sicherstellen kann, dass die Daten ohne Fehler übertragen worden sind, dafür kam dann eine 8Bit Checksumme zum Einsatz. Das Datenpaket besteht immer aus einem Command, Payload und der Checksumme umfasst von * für Master zu Slaves und # für die Kommunikation von Slaves zum Master. In unserem Protokoll hat der Payload immer 32 Byte, der Command immer 4 Byte und die Checksumme immer 3 Byte, außerdem wird noch übertragen von welchem Master das Datenpaket kommt, um zu validieren, dass man nur Daten vom richtigen Master empfängt

Beim Erkennen der Slaves über den Master wird ein bestimmter Command gesendet welcher dann die MAC-Adresse des Geräts zurückgibt und die Adresse des Masters speichert so dass nur Daten von diesem Master angenommen werden dürfen.

Jeder Command wird mit einem OKAY bestätigt oder mit einem NACK abgelehnt um den Master zu signalisieren ob ein Command erfolgreich gesendet, empfangen und verarbeitet wurde. Im Master existiert ein Timer welcher bei nicht erfolgreicher Transmission des Kommandos automatisch den letzten Command erneut sendet.

Das Protokoll unter unserem Layer ist das von Espressif implementierte ESP-NOW⁸ Kommunikationssystem zusammen mit der Implementierung in Micro Python.

Hier noch ein Beispiel wie einer unserer Command aussehen könnte:

*AA:BB:CC:DD:EE:FFCOLR000111222333444555@@@@@@@@@@@@@123
*

Der erste Teil hier rot ist das Start bzw. Endzeichen, danach folgt in Gelb die MAC-Adresse des Geräts welches erreicht werden soll, dann in Grün der Command in blau der Payload und in violett die Checksumme, wie schon angesprochen hat der Command 32 Zeichen, alles Überschüssige wird mit @ Zeichen aufgefüllt.

⁸ <https://docs.micropython.org/en/latest/library/espnow.html>

5 Software Master

Variables (variabels.py)

Die variabels.py-Datei definiert globale Variablen. Diese Variablen steuern verschiedene Sendeoperationen, speichern Zwischenwerte und enthalten Listen von MAC-Adressen der LED-Slave

Flags zur Steuerung der Sendeoperationen (SEARCH_SEND, HRBT_SEND, COLR_SEND, SENS_SEND, TONE_SEND).

Zwischenspeicher für die zu sendenden Daten (COLOR_PAYLOAD, SENS_PAYLOAD, TONE_PAYLOAD).

Listen und Indizes zur Verwaltung von MAC-Adressen (mac_list, NEEDED_MAC_INDEX).

Steuerungs-Flags für die Timer (SEND_ONCE, EXTEND_TIMER).

Bestätigungsflag für empfangene Befehle (COMMAND_ACK).

Hauptprogramm (main.py)

Das main.py-Modul ist das zentrale Programm der Software. Es initialisiert die verschiedenen Module und Timer und enthält Funktionen zur Handhabung der Kommunikation mit dem ESP und dem PC. Es steuert auch die Sendeoperationen basierend auf den gesetzten Flags und verwaltet die empfangenen Befehle.

```
def send_commands(t):  
    if variabels.SEARCH_SEND == 1 and variabels.SEND_ONCE == 0:  
        variabels.SEND_ONCE = 1  
        mac = com.get_mac()  
        command = "SRCH"  
        payload = ""  
        buffer = com_handler.encode_message(mac, command, payload)  
        com.send_message(buffer)
```

Abbildung 3 Beispiel Command

Außerdem werden hier alle Sende und Empfangsoperationen gesteuert basierend auf gesetzten Flags, außerdem werden hier alle Empfangen Befehle weiterverarbeitet. So wie Daten an den Computer gesendet.

```
78     espcom_timer.init(period=100, callback=communicate_with_esp_pc)
79     send_timer.init(period=100, callback=send_commands)
```

Abbildung 4 Timer Init

ESP-NOW Kommunikation (espnwcom.py)

Das Kommunikationsmodul verwaltet die drahtlose Kommunikation über ESP-NOW, ein proprietäres drahtloses Protokoll von Espressif. Dieses Modul ist für die Einrichtung und Handhabung der drahtlosen Datenübertragung verantwortlich.

Initialisierung: Das Modul setzt den WLAN-Modus auf Client, aktiviert ESP-NOW und fügt die Broadcast-MAC-Adresse hinzu, um Nachrichten an alle Geräte im Netzwerk senden, empfangen zu können.

Nachrichteneingang: Es wird kontinuierlich auf eingehende Nachrichten überprüft, die dann decodiert und an das Hauptmodul weitergeleitet werden.

Nachrichtensenden: Nachrichten können an die Broadcast-MAC-Adresse gesendet werden, um alle Geräte im Netzwerk gleichzeitig zu erreichen

```
def send_message(self,payload):
    try:
        self.e.send(self.__bcast_mac, payload)
        return True
    except:
        return False
    else:
        return False
```

Abbildung 5 Beispiel für ESPNOW

Button Handler (button.py)

Das button.py-Modul überwacht die Zustände der Tasten, die mit den GPIO-Pins des Microcontrollers verbunden sind. Es stellt Funktionen zur Verfügung, um den Zustand der Tasten auszulesen und in einem definierten Format an den PC zurückzugeben.

```

# Initialize a list to hold the button states
states = []
# Check each button and append '1' or '0' to the states list
for button in self.buttons:
    state = '1' if button.value() else '0'
    states.append(state)
# Join the states list into a string separated by '$' and return
return '$'.join(states)

```

Abbildung 6 Funktion welche den Status-String für die Buttons erstellt

Befehlshandling (command_handler.py)

Dieses Modul ist verantwortlich für die Verarbeitung eingehender Befehle und die Ausführung entsprechender Aktionen, hauptsächlich werden hier flags gesetzt, und Commands auf das Senden und Empfangen vorbereitet

Befehlsdecodierung: Eingehende Nachrichten werden auf ihre Gültigkeit überprüft und decodiert. Dazu gehört die Überprüfung der Prüfsumme, um sicherzustellen, dass die Nachricht während der Übertragung nicht verändert wurde.

Befehlshandlung: Basierend auf den empfangenen Befehlen werden verschiedene Aktionen ausgeführt, wie z.B. das Umschalten der LEDs, das Abrufen und Senden von Sensordaten oder das Abspielen von Tönen über den Buzzer.

```

def decode_message(self,message):
    if message[0] != '#' or message[-1] != '#':
        raise ValueError("Message should start and end with '#')

    source = message[1:18]
    command = message[18:22]
    payload = message[22:54].rstrip('@')
    received_checksum = int(message[54:56], 16)

    data_to_check = message[1:54]
    calculated_checksum = self.xor_checksum(data_to_check.encode('utf-8'))

    if received_checksum != calculated_checksum:
        raise ValueError("Checksum does not match")

    return source, command, payload

```

Abbildung 7 Message Decoder

PC-Kommunikation (pcCOM.py)

Das pcCOM.py-Modul verwaltet die serielle Kommunikation zwischen dem Microcontroller und einem PC. Es enthält Funktionen zum Senden und Empfangen von Befehlen, zur Berechnung von Prüfsummen und zur Kodierung und Dekodierung von Nachrichten. Es überwacht auch die Zustände der Tasten und verarbeitet Befehle entsprechend. Die Commands sind hierbei sehr ähnlich zu denen welche an die Lampen gesendet werden, nur ohne die MAC-Adresse.

```
if self.uart.any():
    message = self.uart.read(41).decode('utf-8')
    if message[0] == '#' and message[-1] == '#':
        command = message[1:5]
        payload = message[5:37]
        received_checksum = message[37:40]
        calculated_checksum = self.calculate_checksum(command + payload)
        if received_checksum == calculated_checksum:
            return command, payload.rstrip('@')
        else:
            raise ValueError("Checksum does not match.")
    else:
        raise ValueError("Invalid message format.")
return None, None
```

Abbildung 8 Funktion für den Empfang von Daten über UART

Funktionsweise der Software

Zunächst werden alle notwendigen Module und Timer initialisiert. Dies geschieht im Hauptprogramm (main.py), dass die Kernfunktionen der Software zusammenführen. Hier werden Instanzen für die ESP-NOW Kommunikation, das Befehlshandling und die PC-Kommunikation erstellt. Zudem werden Timer eingerichtet, die regelmäßig bestimmten Funktionen aufrufen, um die Kommunikation und die Steuerung der LED-Lampe aufrechtzuerhalten.

Die Zustände der Tasten, die mit den GPIO-Pins des Microcontrollers verbunden sind, werden kontinuierlich überwacht. Das button.py-Modul liest diese Zustände aus und

gibt sie in einem definierten Format zurück. Diese Informationen können dann verwendet werden, um entsprechende Aktionen basierend auf den Tastendrücken auszulösen.

Die Kommunikation zwischen den Geräten erfolgt über ESP-NOW und UART. Das `espnwcom.py`-Modul verwaltet die drahtlose Kommunikation über ESP-NOW. Es initialisiert die WLAN-Module, verwaltet die Peers und ermöglicht das Senden und Empfangen von Nachrichten. Über das `pcCOM.py`-Modul wird die serielle Kommunikation zwischen dem Microcontroller und einem PC abgewickelt. Es stellt sicher, dass Befehle und Daten korrekt gesendet und empfangen werden, und überwachen die Zustände der Tasten, um auf entsprechende Befehle vom PC zu reagieren.

Das `command_handler.py`-Modul ist für die Verarbeitung der empfangenen Befehle zuständig. Es decodiert die Nachrichten, überprüft die Prüfsummen und führt die entsprechenden Aktionen aus. Dazu gehört die Verarbeitung von spezifischen Befehlen wie Herzschlag-Signalen, Farbeinstellungen und Sensor-Daten. Die Sensordaten werden codiert und in ein definiertes Format umgewandelt, das von anderen Teilen der Software weiterverarbeitet werden kann.

Die Timer im Hauptprogramm steuern verschiedene Sendeoperationen zur Lampe und zum Host PC. Sie rufen regelmäßig Funktionen auf, die überprüfen, ob neue Nachrichten empfangen wurden, und ob bestimmte Befehle gesendet werden müssen, auch die Empfangsoperationen werden hier von einem Timer verwaltet.

6 Software Slave

Hauptmodul (`main.py`)

Das Hauptmodul ist das Herzstück der Software und initialisiert die verschiedenen Komponenten des Systems, einschließlich der LEDs, der Kommunikationsschnittstelle, des Buzzer-Moduls und der Sensoren. Es führt eine Endlosschleife aus, in der es kontinuierlich auf eingehende Nachrichten überprüft und diese verarbeitet.

Initialisierung: Zu Beginn werden alle relevanten Pins für die LEDs, den Buzzer und die I2C-Verbindung für die Sensoren initialisiert.

```

while True:
    if com.check_received():
        try:
            received_message = com.receive_message()
            source, command, payload = com_handler.decode_message(received_message)
            com_handler.handle_command(source, command, payload)
        except:
            print("MESSAGE TO OTHER DEVICE OR ERROR")

```

Abbildung 9 Loop ohne Timer da hier nicht auf UART gewartet werden muss

Endlosschleife: In einer fortlaufenden Schleife wird auf eingehende Nachrichten über die ESP-NOW-Schnittstelle überprüft. Wenn Nachrichten empfangen werden, werden diese zur weiteren Verarbeitung an das Kommandohandler-Modul weitergeleitet.

Kommunikationsmodul (espnowcom.py)

Das Kommunikationsmodul verwaltet die drahtlose Kommunikation über ESP-NOW, ein proprietäres drahtloses Protokoll von Espressif. Dieses Modul ist für die Einrichtung und Handhabung der drahtlosen Datenübertragung verantwortlich.

Initialisierung: Das Modul setzt den WLAN-Modus auf Client, aktiviert ESP-NOW und fügt die Broadcast-MAC-Adresse hinzu, um Nachrichten an alle Geräte im Netzwerk senden, empfangen zu können.

Nachrichtenempfang: Es wird kontinuierlich auf eingehende Nachrichten überprüft, die dann decodiert und an das Hauptmodul weitergeleitet werden.

Nachrichtensenden: Nachrichten können an die Broadcast-MAC-Adresse gesendet werden, um alle Geräte im Netzwerk gleichzeitig zu erreichen.

Das Modul wurde aus der Software des Masters kopiert.

LED-Steuermodul (ledControl.py)

Das LED-Steuermodul ermöglicht die Steuerung der LEDs der Lampe über PWM. Dieses Modul ist zentral für die Anpassung der Helligkeit und Farbe der LEDs verantwortlich.

Initialisierung: Die PWM-Frequenz, die Timer Kanäle und der Start Duty Cycle wird gesetzt.

Hex-Code-Verarbeitung: Das Modul kann Hex-Codes in PWM-Duty-Zyklen umwandeln, um die für die Hardware Ansteuerung benötigten Werte von 0-255 zu erhalten

LED-Steuerung: Es ermöglicht das dimmen der LEDs sowie die Validierung von Hex-Codes, um sicherzustellen, dass die Eingaben korrekt und im erwarteten Format sind.

```
def validate_hex_code(self, hex_code):
    """Validate the hex code format."""
    if not isinstance(hex_code, str):
        raise ValueError("Hex code must be a string.")
    if not hex_code.startswith('0x') and not hex_code.startswith('0X'):
        raise ValueError("Hex code must start with '0x' or '0X'.")
    hex_str = hex_code[2:]
    if len(hex_str) != 12:
        raise ValueError("Invalid hex code length. Expected 12 characters.")
    try:
        int(hex_str, 16)
    except ValueError:
        raise ValueError("Hex code contains invalid characters.")

def hex_to_duty(self, hex_code):
    """Convert hex code to PWM duty values."""
    self.validate_hex_code(hex_code)
    hex_str = hex_code[2:]
    duties = [int(hex_str[i:i+2], 16) for i in range(0, 12, 2)]
    mapped_duties = [int(d * 1023 / 255) for d in duties] # Map to PWM range
    return mapped_duties

def set_leds(self, hex_code):
    """Set the LEDs to the brightness values specified by the hex code."""
    duties = self.hex_to_duty(hex_code)
    for led, duty in zip(self.leds, duties):
        led.duty(duty)
```

Abbildung 10 Die wichtigsten Funktionen zum Steuern der LED

Sensorsteuerungsmodul (sensorControl.py)

Dieses Modul verwaltet die verschiedenen Sensoren, die über I2C verbunden sind, einschließlich eines Temperatur- und Feuchtigkeitssensors (AHT20), eines TVOC-Sensors (AGS10) und eines Lichtsensors (LTR308).

Initialisierung: Konfiguriert den I2C-Bus und initialisiert die Sensoren, um sicherzustellen, dass sie betriebsbereit sind.

Datenabruf: Liest kontinuierlich Daten von den Sensoren und erstellt einen Payload-String zur Übertragung. Diese Daten können zur Steuerung der LED-Lampe oder zur Überwachung der Umgebung verwendet werden.

```
def get_sensor_data(self):
    """Read data from all sensors and generate a payload string"""
    # Read temperature from AHT20 sensor
    temperature = self.aht20_sensor.temperature
    # Read humidity from AHT20 sensor
    humidity = self.aht20_sensor.relative_humidity

    # Read TVOC from AGS10 sensor
    tvoc = self.ags10_sensor.total_volatile_organic_compounds_ppb

    # Read lux (ambient light) from LTR308 sensor if new data is available
    if self.ltr_sensor.new_data_available():
        lux = self.ltr_sensor.read_lux()
    else:
        lux = None

    # Generate payload string with sensor data
    payload = f"{temperature:.2f}${humidity:.2f}${tvoc}"
    if lux is not None:
        payload += f"${lux}"

    return payload # Return the payload string
```

Abbildung 11 Funktion welche die Sensordaten erzeugt

Buzzer-Steuermodul (soundControl.py)

Das Buzzer-Modul steuert einen piezoelektrischen Buzzer, um Töne zu erzeugen. Es wird hauptsächlich für akustische Signale und Alarmer verwendet.

Initialisierung: Der Buzzer wird konfiguriert, um verschiedene Töne und Frequenzen erzeugen zu können.

Song-Management: Das Modul kann verschiedene vordefinierte Lieder speichern und abspielen, was für akustische Benachrichtigungen nützlich ist.

Tonausgabe: Töne werden anhand von Frequenztabelle abgespielt, um die gewünschten akustischen Signale zu erzeugen.

```
def play_song(self, name):
    # Play a song by name if it exists in the dictionary
    if name not in self.songs:
        return False # Return False if the song is not found

    notes = self.songs[name] # Get the list of notes for the song
    for note, duration in notes:
        freq = self.NOTE_FREQS.get(note, 0) # Get the frequency for the note
        if freq == 0:
            self.buzzer.duty(0) # Turn off the buzzer for a pause
        else:
            self.buzzer.freq(freq) # Set the buzzer frequency
            self.buzzer.duty(512) # Set the duty cycle to play the note
            time.sleep(duration) # Wait for the duration of the note
    self.buzzer.duty(0) # Turn off the buzzer after the song is finished
```

Abbildung 12 Funktion um einen Sound abzuspielen

Befehlshandlermodul (command_handler.py)

Dieses Modul ist verantwortlich für die Verarbeitung eingehender Befehle und die Ausführung entsprechender Aktionen, die hauptsächlich die Steuerung der LED-Lampe und das Abrufen der Sensordaten betreffen.

Befehlsdecodierung: Eingehende Nachrichten werden auf ihre Gültigkeit überprüft und decodiert. Dazu gehört die Überprüfung der Prüfsumme, um sicherzustellen, dass die Nachricht während der Übertragung nicht verändert wurde.

Befehlshandlung: Basierend auf den empfangenen Befehlen werden verschiedene Aktionen ausgeführt, wie z.B. das Umschalten der LEDs, das Abrufen und Senden von Sensordaten oder das Abspielen von Tönen über den Buzzer.

Funktionsweise der Software

Die Software läuft in einer Endlosschleife und überwacht kontinuierlich eingehende Nachrichten über ESP-NOW. Bei Empfang einer Nachricht wird diese decodiert und an das Command-Handler-Modul weitergeleitet, welches dann entsprechende Aktionen ausführt. Diese Aktionen umfassen das Einstellen der LED-Farben, das Abrufen und Senden von Sensordaten sowie das Abspielen von Tönen über den Buzzer.

Die Sensoren liefern kontinuierlich Daten über Temperatur, Luftfeuchtigkeit, TVOC (Total Volatile Organic Compounds) und Umgebungslicht. Diese Daten werden aktuell nur an den Master weitergeben und dort ausgegeben, aktuell werden diese noch nicht weiterverarbeitet.

Die Module sind so konzipiert, dass sie leicht erweiterbar und modifizierbar sind, nach Baukastenprinzip.

Für alle verwendeten Sensoren musste eine eigene Library geschrieben werden da es leider keine fertige zum Download gab, beim Schreiben der Librarys gab es einige Probleme mit einigen Commands über den I2C Bus, diese konnten aber durch genaues lesen des Datenblatts gelöst werden (ich habe ein Bit gesetzt was ich nicht durfte). Die Librarys sind zu finden unter `ags10.py`, `ahtx0py` und `ltr308.py`.

```
def read_ambient_light(self):
    # Read the ALS data registers
    data0 = self.i2c.readfrom_mem(self.address, self.REG_ALS_DATA_0, 1)[0]
    data1 = self.i2c.readfrom_mem(self.address, self.REG_ALS_DATA_1, 1)[0]
    data2 = self.i2c.readfrom_mem(self.address, self.REG_ALS_DATA_2, 1)[0]

    # Combine the data bytes into a single value
    als_data = (data2 << 16) | (data1 << 8) | data0

    return als_data
```

Abbildung 13 Beispiel Funktion aus der LTR308 library

7 Probleme und Verbesserungen

I. Hardware:

- a. Leider ist uns beim Designer der Elektronik der Fehler unterlaufen, dass der Temperatursensor die Temperatur der Platine und nicht, dass der Umgebung misst, hier muss noch eine Thermische Trennung erfolgen
- b. Die Anschlussklemmen für den Strom sind nicht ideal, diese müssten auch getauscht werden
- c. Auch wäre ein Button an diesem Gerät sehr praktisch gewesen.
- d. Beim Buzzer wurde ein MOSFET verbaut welcher leider Aufgrund seiner Körperdiode Probleme verursacht hat (Buzzer sehr leise) also wurde dieser durch einen Transistor ersetzt

II. Software Master

- a. Hier muss auf jeden Fall noch an der Asynchronität gearbeitet werden, diese ist aktuell eher marginal, hierfür müsste die ESP-NOW Implementation und auch die USB-Schnittstelle mit ayncio implementiert werden.
- b. Es sollte eine Verschlüsselung auf dem Master eingebaut werden um die Nachrichten zwischen dem Master und Slave zu verschlüsseln.
- c. Es sollten noch einige Fehler abgefangen werden, wenn falsche Daten eingegeben werden
- d. Die Kommunikation zwischen PC und Master sollte in Zukunft über Funk erfolgen

III. Software Slave

- a. Hier muss auf jeden Fall noch an der Asynchronität gearbeitet werden
- b. Auch wieder die Verschlüsselung sollte, wenn man an diesem Projekt weitermacht noch hinzugefügt werden

8 Fazit

Mir hat das Projekt zusammen mit Justin sehr viel Spaß gemacht, ich habe trotz meiner jetzt doch schon einigen Erfahrung immer noch etwas Neues über Python lernen können. Ich habe auch noch einiges über Micro Python auf dem ESP32 gelernt. Außerdem dass Buzzer nicht mit MOSFETs funktionieren, und dass die Thermischen Schlitze doch sehr relevant sind. Auch die Vorstellung, dass man, dass Ding fertig machen

könnte und dann verkaufen fand ich gut, und ich hätte es gerne auch fertig gemacht. Leider konnten wir aus Zeitgründen nicht alles was wir uns überlegt haben auch umsetzen aber daran kann man ja auch nochmals in Zukunft arbeiten. Gerne hätte ich noch die GUI von Justin zum Laufen bekommen aber wie gesagt dafür war keine Zeit mehr. Persönlich hat mir es auch Spaß gemacht Justin noch ein paar Tipps zu geben und mein Wissen weiterzugeben.

9 Quellenverzeichnis

AGS10. (Mai 2024). Von https://www.lcsc.com/product-detail/Temperature-and-Humidity-Sensor_Aosong-Guangzhou-Elec-AHT20_C2757850.html abgerufen

AHT20. (Mai 2024). Von https://www.lcsc.com/product-detail/Temperature-and-Humidity-Sensor_Aosong-Guangzhou-Elec-AHT20_C2757850.html abgerufen

CN5711 Constant Current Source. (Mai 2024). Von https://www.lcsc.com/product-detail/LED-Drivers_ShangHai-Consonance-Elec-CN5711_C35634.html abgerufen

ESP32-S3 Datasheet. (Mai 2024). Von https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf#page=26&zoom=100,76,76 abgerufen

ESP32-S3 Technical Refrence Manual. (Mai 2024). Von https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf abgerufen

LTR308. (Mai 2024). Von https://www.lcsc.com/product-detail/Ambient-Light-Sensors_Lite-On-LTR-308ALS-01_C492382.html abgerufen

MicroPython Dokumentation. (Mai 2024). Von <https://docs.micropython.org/en/latest/esp32/quickref.html> abgerufen

Steinigke. (Mai 2024). Von <https://www.steinigke.de/de/mpne6510838-led-hcl-10w-rgbwauv-led-tmh-46-moving-head-wash-hl-rgbwauv-6ine1.html> abgerufen

10 Abbildungsverzeichnis

Abbildung 1 Platine Unterseite	5
Abbildung 2 Platine Oberseite	5
Abbildung 3 Beispiel Command.....	7
Abbildung 4 Timer Init.....	8
Abbildung 5 Beispiel für ESPNOW	8
Abbildung 6 Funktion welche den Status-String für die Buttons erstellt.....	9
Abbildung 7 Message Decoder.....	9
Abbildung 8 Funktion für den Empfang von Daten über UART	10
Abbildung 9 Loop ohne Timer da hier nicht auf UART gewartet werden muss.....	12
Abbildung 10 Die wichtigsten Funktionen zum Steuern der LED.....	13
Abbildung 11 Funktion welche die Sensordaten erzeugt.....	14
Abbildung 12 Funktion um einen Sound abzuspielen.....	15
Abbildung 13 Beispiel Funktion aus der LTR308 library	16
Abbildung 14 UML Slave	20
Abbildung 15 UML Master	20

11 Anhang



Abbildung 15 UML Master

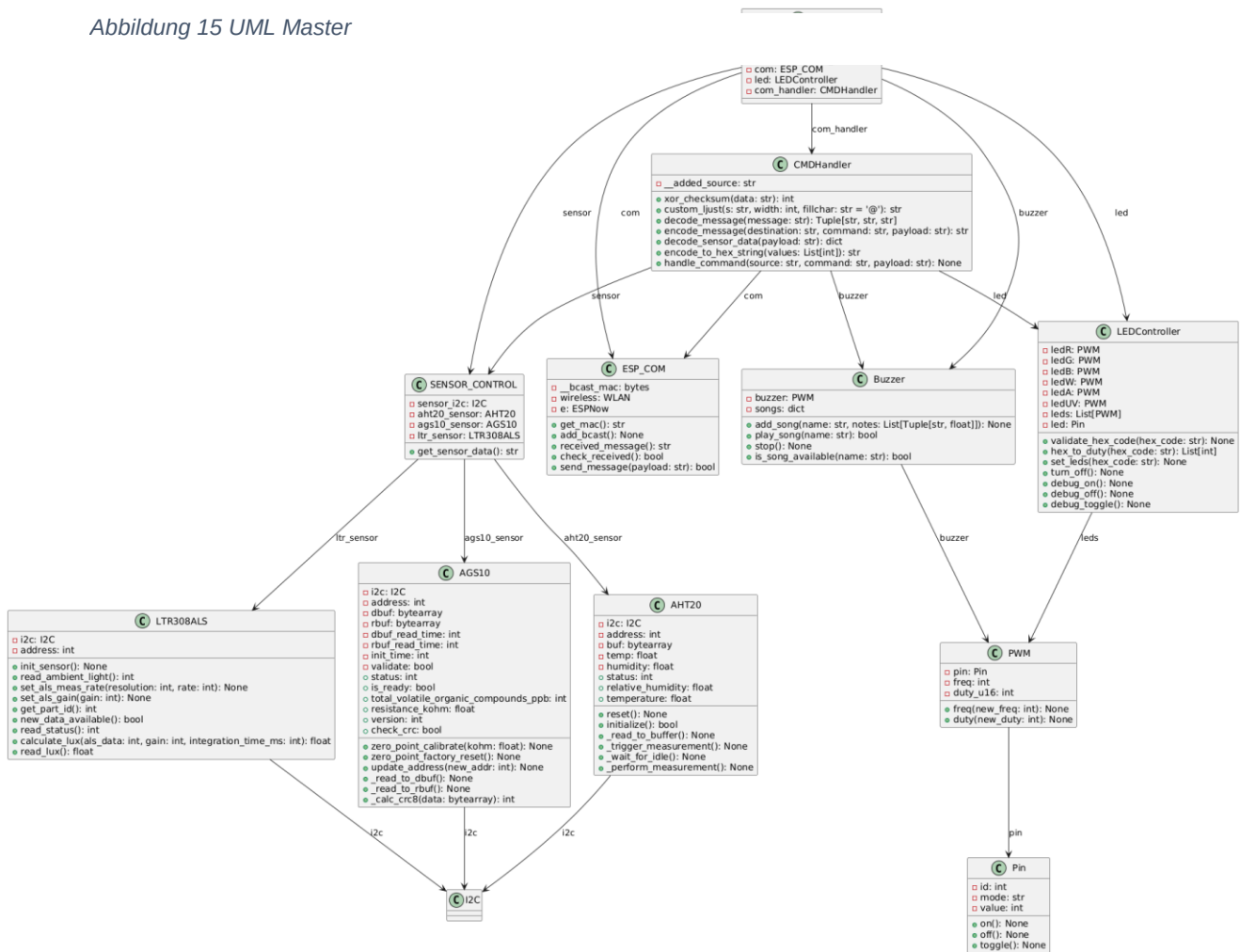


Abbildung 14 UML Slave

